

Technical Deep Dive: Architectures and Implementation Dynamics of Generative Novel View Synthesis with 3D-Aware Diffusion Models (GeNVS)

Executive Summary

This technical report provides an exhaustive analysis of the architectural methodology presented in "Generative Novel View Synthesis with 3D-Aware Diffusion Models" (GeNVS) by Chan et al., presented at ICCV 2023. This document is specifically engineered to support the development of a high-fidelity PyTorch implementation, offering a granular dissection of the system's four critical subsystems: the **Geometry Encoder**, the **3D Feature Volume**, the **Neural Renderer**, and the **Denoising Diffusion Backbone**.

The synthesis of novel views from sparse inputs has historically been bifurcated into two distinct paradigms: regression-based methods and generative approaches. Regression methods, exemplified by PixelNeRF, utilize explicit geometric structures to ensure multi-view consistency but frequently produce blurry, low-frequency outputs in occluded regions due to the inherent uncertainty of the regression objective (MSE loss). Conversely, generative methods, particularly Denoising Diffusion Probabilistic Models (DDPMs), excel at hallucinating high-frequency photorealistic details but often fail to maintain rigorous 3D consistency across changing viewpoints, leading to geometric drift and temporal flickering.

GeNVS bridges this divide by proposing a hybrid architecture. It does not discard the explicit geometry of regression methods; rather, it repurposes it. Instead of regressing RGB colors directly, the system regresses a **latent 3D feature field**. This field is rendered into a 2D "feature image" that serves as a structural conditioning signal for a 2D diffusion model. This crucial design choice means the diffusion model is no longer tasked with inventing geometry from scratch; it acts as a "neural renderer" that paints high-fidelity texture onto a consistent geometric scaffolding provided by the feature volume.¹

A primary focus of this report is to clarify the architectural nature of the denoising backbone. While often colloquially associated with "3D" generation, the GeNVS system employs a **2D U-Net** for the diffusion process, not a 3D U-Net. The "3D" aspect is encapsulated entirely within the feature volume and volumetric rendering modules. This distinction is paramount for implementation, as it dictates memory allocation, tensor dimensionality, and computational complexity. The report details the specific configuration of this 2D U-Net, which is a lightweight implementation (approximately 90 million parameters) of the Ablated Diffusion Model (ADM) architecture, conditioned via channel-wise concatenation rather than

cross-attention.³

The following sections provide a complete reconstruction of the model's logic, supported by mathematical formulations of the rendering equations, tensor shape analysis for memory planning, and comparative insights into the design choices (e.g., ResNet-34 vs. Transformers, Stratified Sampling vs. Importance Sampling). This deep dive aims to equip the engineering team with the theoretical and practical context required to replicate the state-of-the-art performance reported in the original study.

1. Introduction and Theoretical Framework

The problem of Novel View Synthesis (NVS) asks a fundamental question: Given a set of source images $\{I_s\}$ and their camera poses $\{P_s\}$, can we synthesize a new image I_t from a target pose P_t that is indistinguishable from a real photograph?

1.1 The Regression-Generation Spectrum

To understand the architectural decisions in GeNVS, one must situate it within the broader spectrum of NVS solutions.

Regression-Based Approaches:

Methods like PixelNeRF and Stereo Magnification treat NVS as a regression problem. They train a network to minimize the mean squared error (MSE) between the rendered pixel and the ground truth.

- **Pros:** By explicitly modeling 3D geometry (via cost volumes or neural fields), these methods guarantee that if a point exists in 3D space, it will project correctly to all views.
- **Cons:** MSE is a unimodal loss function. In regions of uncertainty (e.g., behind an object), the optimal minimization strategy is to output the mean of all possible pixel values, resulting in a distinct "blur." They cannot hallucinate plausible details for unseen areas.⁵

Generative Approaches:

Methods like 3DiM or Zero-1-to-3 utilize diffusion models to generate pixels. They treat the target view as a sample from a conditional probability distribution $p(I_t|I_s, P_s, P_t)$.

- **Pros:** Diffusion models optimize a variational lower bound on the data likelihood, allowing them to sample from multimodal distributions. This means they can generate sharp, realistic details even in occluded regions (e.g., imagining the back of a chair).
- **Cons:** Without explicit 3D constraints, pure 2D diffusion models often "hallucinate" geometry that contradicts the input views or varies wildly as the camera moves, leading

to the "Janus problem" (multi-faced objects) or flickering.⁴

1.2 The GeNVS Hybrid Paradigm

GeNVS proposes that the solution lies in the intersection of these fields. The architecture is effectively split into two distinct stages that are trained end-to-end:

1. **The Geometric Stage (Encoder + Renderer):** This stage builds a coarse, explicit 3D representation of the scene based on the input view. It creates a "Feature Volume" that captures the scene's rough structure and semantics.
2. **The Generative Stage (Diffusion U-Net):** This stage takes the coarse, rendered projection of the geometry and refines it. It uses the generative prior of the diffusion model to fill in textures and resolve ambiguities, strictly adhering to the structural guidance provided by the first stage.

This framework shifts the burden of "3D consistency" to the feature volume and "photorealism" to the diffusion model.

1.3 Core Mathematical Formulation

The goal is to sample from the conditional distribution:

$$\mathbf{x}_{target} \sim p_{\theta}(\mathbf{x} | \mathbf{x}_{input}, P_{input}, P_{target})$$

GeNVS models this via a diffusion process where the denoising network ϵ_{θ} is conditioned on a rendered feature image F :

$$\mathcal{L}_{simple} = \mathbb{E}_{t, \mathbf{x}_0, \epsilon} [\|\epsilon - \epsilon_{\theta}(\mathbf{x}_t, t, F)\|^2]$$

where F is the result of a differentiable volume rendering process π :

$$F = \pi(E(\mathbf{x}_{input}), P_{input}, P_{target})$$

Here, E is the encoder and π is the volumetric renderer. The entire pipeline is differentiable, allowing gradients from the pixel-space diffusion loss to backpropagate all the way through the renderer to the geometry encoder, forcing the encoder to learn features that—when rendered—provide the most useful scaffolding for the denoiser.²

2. The Geometry Encoder: From Pixels to Frustums

The Encoder is the foundational component of the GeNVS pipeline. Its responsibility is to

extract a high-dimensional feature representation from the source image that can be "lifted" into 3D space. Unlike standard image classifiers that pool information into a vector, this encoder must preserve precise spatial correspondences.

2.1 Backbone Architecture: ResNet-34

The choice of backbone is a critical engineering decision. The GeNVS authors explicitly select **ResNet-34**.⁴

2.1.1 Rationale for Selection

While ResNet-50 and ResNet-101 are more powerful in terms of representational capacity, they are computationally heavier and significantly deeper. In the context of 3D feature lifting:

1. **Receptive Field vs. Locality:** Very deep networks often have receptive fields that span the entire image, leading to "global" features. However, for 3D reconstruction, local features (edges, textures) are paramount. ResNet-34 provides a balance, offering sufficient semantic abstraction without completely diffusing local geometric cues.
2. **Memory Constraints:** The encoder is just the first step. The subsequent creation of a 3D volume consumes massive amounts of VRAM. Using a lighter backbone like ResNet-34 frees up memory for the volumetric operations, which scale cubically.
3. **Pretraining:** The model is initialized with ImageNet weights, providing a robust starting point for texture and object recognition.⁹

2.2 DeepLabV3+ and Multi-Scale Context

The backbone feeds into a **DeepLabV3+** architecture.⁴ DeepLab is traditionally a semantic segmentation architecture. Its selection here is insightful: NVS can be viewed as a "semantic segmentation" task where the "classes" are continuous feature descriptors of 3D geometry.

2.2.1 Atrous Spatial Pyramid Pooling (ASPP)

The core of DeepLabV3+ is the ASPP module. Standard convolutions have a fixed receptive field. ASPP applies convolutions with different **dilation rates** in parallel.

- **Mechanism:** It probes the feature map at multiple scales effectively capturing both small details (e.g., the leg of a chair) and large contexts (e.g., the shape of the room) simultaneously.
- **Relevance to NVS:** This is crucial for resolving depth ambiguity. Determining the depth of a smooth wall requires large-scale context (corners of the room), while determining the depth of a thin wire requires fine-scale context. ASPP provides both.

2.3 Structural Modifications for Geometry

Standard DeepLabV3+ implementations are designed to output class logits (probabilities). For GeNVS, the architecture is modified to output continuous, high-fidelity geometric features.

2.3.1 Learnable Upsampling

A standard DeepLab decoder often uses bilinear upsampling to restore spatial resolution (e.g., upsampling by factor 4 or 8 to match input size).

- **The Problem:** Bilinear interpolation is a fixed, non-learnable operation. It "smears" features, blurring sharp depth discontinuities. In NVS, a sharp edge represents a depth jump; blurring this edge causes "flying pixels" or fog artifacts in the 3D rendering.
- **The Solution:** GeNVS replaces bilinear upsampling with **learnable upsampling blocks**.⁴ These are typically implemented as ConvTranspose2d layers (Transposed Convolutions) or pixel-shuffle operations followed by convolutions. This allows the network to learn *how* to upsample features to preserve edges, effectively learning a super-resolution operator for the feature maps.

2.3.2 Deterministic Normalization

The research highlights a specific configuration for normalization layers:

- **Batch Normalization (BN):** Disabled/Removed in the feature encoder.
- **Dropout:** Disabled.
- **Reasoning:** The encoder must act as a deterministic mapping $\mathcal{T}(\mathbf{x})$. If the encoder output varies stochastically (as it would with Dropout or fluctuating Batch Norm stats during training), the 3D volume would "jitter." The diffusion model, which relies on this volume as a stable anchor, would interpret this jitter as noise, destabilizing the training. The encoder must be a rigid geometric estimator.⁴

2.4 Tensor Dimensionality and Reshaping

The output of the encoder is a 2D feature map that implicitly contains the 3D structure.

- **Input Image:** $B \times 3 \times 128 \times 128$
- **Encoder Output:** The encoder is designed to output a tensor with a massive channel dimension. specifically, it outputs $C_{total} = C_{feat} \times D$ channels.
 - $C_{feat} = 16$ (Feature size per voxel)
 - $D = 64$ (Number of depth planes)
 - Output Shape: $B \times (16 \times 64) \times 128 \times 128 = B \times 1024 \times 128 \times 128$.
- **Reshaping:** This 2D tensor is conceptually "unfolded" along the channel dimension to create the 3D volume.
 - Reshaped: $B \times 16 \times 64 \times 128 \times 128$.
 - Dimensions: (Batch, Feature, Depth, Height, Width).

This reshaping operation effectively "lifts" the 2D features into 3D space, assuming that the

channels of the 2D map correspond to features at different depths along the camera ray.⁴

3. The 3D Feature Volume: Representing Latent Geometry

The 3D Feature Volume is the central data structure that facilitates 3D consistency. It serves as an intermediate representation (IR) that decouples the source view from the target view.

3.1 Frustum-Aligned Grids vs. Euclidean Voxel Grids

Most classical 3D reconstruction methods utilize a Euclidean voxel grid (a cube fixed in world space). GeNVS, following the precedent of PixelNeRF, utilizes a **Frustum-Aligned Grid**.

- **Definition:** The volume is defined within the view frustum of the *source* camera. It is not an axis-aligned box in the world; it is a perspective-warped wedge.
- **Coordinate System:** The grid is parameterized by Normalized Device Coordinates (NDC) (u, v) and depth d .
 - (u, v) : Pixel coordinates in the source image (ranging -1 to 1).
 - d : Depth or disparity along the ray (ranging from z_{near} to z_{far}).

Why Frustum Alignment?

1. **Efficiency:** A Euclidean grid wastes vast amounts of memory representing empty space (sky, air). A frustum grid places resolution exactly where the camera is looking.
2. **Alignment:** The encoder operates in 2D image space. Mapping 2D pixels (u, v) directly to frustum voxels (u, v, d) maintains spatial alignment. A pixel at $(100, 100)$ in the image corresponds to the "tube" of voxels at $(100, 100, :)$ in the volume. This makes the "lifting" operation trivial (simple reshaping).

3.2 Volume Specifications and Memory Complexity

The specific dimensions chosen for GeNVS are a careful trade-off between fidelity and memory.

Dimension	Value	Description	Implication
Channels (C_{feat})	16	Latent size per voxel	Compressed representation of

			color + density. Small enough to fit in VRAM, large enough to encode semantics. ⁴
Depth (D)	64	Depth planes	Defines the Z-resolution. 64 planes is relatively coarse, implying the model relies on the diffusion upsampler for fine depth details. ⁴
Spatial (H, W)	128	Grid resolution	Matches input image resolution. Ensures 1:1 pixel-to-ray mapping during lifting. ⁴

Memory Analysis:

A single feature volume for one image in a batch has dimensions $16 \times 64 \times 128 \times 128$.

- Total elements: $16 \times 64 \times 128 \times 128 \approx 16.7$ million elements.
- Size (Float32): $16.7\text{M} \times 4 \text{ bytes} \approx 67 \text{ MB}$.
- With a batch size of 4 and gradients (requires storing activations), this quickly escalates into gigabytes of VRAM. This necessitates the use of memory-efficient backbones (ResNet-34) and reduced-resolution rendering (discussed in Section 4).

3.3 Multi-View Aggregation Strategy

A robust NVS system must handle multiple input images to resolve occlusions. If we have N source images, we have N distinct frustum feature volumes $W_1, \dots, W_N$.

GeNVS does **not** fuse these into a single global volume (which would require complex resampling and registration). Instead, it keeps them separate and aggregates them **during the rendering phase**.

- **Logic:** The "scene" is represented as a collection of local frustum volumes.
- **Querying:** When rendering a ray for a target view, the system queries *all* overlapping

frustum volumes.

- **Aggregation Operator:** The features retrieved from each volume are averaged.

$$\bar{w}(\mathbf{x}) = \frac{1}{|S|} \sum_{i \in S} w_i(\mathbf{x})$$

where S is the set of source views that observe point $\mathbf{x}$. This "mean pooling" is permutation invariant (order of input images doesn't matter) and handles varying numbers of inputs gracefully.³

4. The Neural Renderer: Differentiable Projection

The Neural Renderer is the differentiable engine that projects the 3D feature volumes onto the 2D target view. It functions similarly to a NeRF renderer but operates on latent feature vectors instead of RGB colors.

4.1 Ray Marching Implementation

For a target camera pose P_{target} and intrinsics K :

1. **Ray Generation:** Generate rays $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ for every pixel in the target image.
2. **Sampling:** Sample $K = 64$ points along each ray. The sampling strategy is **Stratified Sampling**:

$$t_k \sim \mathcal{U}[t_{near} + \frac{k-1}{K}(t_{far} - t_{near}), t_{near} + \frac{k}{K}(t_{far} - t_{near})]$$

Why Stratified? Randomizing sample positions within bins adds noise during training, which acts as a regularizer and prevents the model from overfitting to a fixed sampling grid. GeNVS explicitly eschews "Hierarchical Importance Sampling" (used in NeRF) to reduce computational overhead, relying on the single-pass volume query for speed.⁴

4.2 Feature Interpolation

For a sample point $\mathbf{x}$ in world space:

1. **Transform:** Project $\mathbf{x}$ into the coordinate system of the source camera i :
 $\mathbf{x}_{cam} = P_i^{-1} \mathbf{x}$.
2. **Project:** Apply intrinsics to find coordinates (u, v, d) in the source frustum.
3. **Sample:** Use **trilinear interpolation** (`torch.grid_sample`) to fetch the feature vector w_i from volume W_i .

- If the point projects outside the source frustum, the feature is zero or masked.
4. **Aggregate:** Average the features from all valid source volumes to get $\bar{w}$.

4.3 The Decoding MLP (f)

The raw feature vector $\bar{w}$ is latent. It needs to be interpreted into geometric density and semantic features. A lightweight MLP performs this decoding at every sample point.

- **Input:** Averaged feature vector $\bar{w} \in \mathbb{R}^{16}$.
- **Architecture:**
 - Layer 1: Linear(16 $\rightarrow$ 64) + ReLU
 - Layer 2: Linear(64 $\rightarrow$ 64) + ReLU
 - Layer 3: Linear(64 $\rightarrow$ 17)
- **Outputs:**
 - Scalar τ : Density (logit).
 - Vector $\mathbf{c}_{feat}$: Feature content (16 dimensions).
- **Skip Connection:** A residual connection adds the input $\bar{w}$ to the output $\mathbf{c}_{feat}$. This helps preserve information from the encoder directly to the renderer output.³

4.4 The Rendering Equation

The renderer integrates the samples along the ray to produce the final pixel value. This is the standard volumetric rendering equation, discretized:

$$C(\mathbf{r}) = \sum_{k=1}^K T_k (1 - \exp(-\text{softplus}(\tau_k) \delta_k)) \cdot \mathbf{c}_{feat,k}$$

- T_k : Transmittance (probability the ray travels to step k without hitting anything).
- **Output:** The result $C(\mathbf{r})$ is a **16-channel feature vector**, not an RGB color. This vector represents the "neural image" F seen from the target viewpoint.

4.5 Performance Optimization: Low-Resolution Rendering

Rendering is the bottleneck. Casting 64 rays for 128×128 pixels involves over 1 million MLP evaluations per image. To mitigate this, GenVS employs a resolution trick:

- **Render Resolution:** The feature image F is rendered at 64×64 (half resolution).
- **Upsampling:** The result is bilinearly upsampled to 128×128 before entering the

diffusion model.

- **Impact:** This reduces rendering cost by 75% (from 16k rays to 4k rays per image), enabling larger batch sizes and faster training without significantly degrading the geometric conditioning, as the diffusion model is adept at super-resolving the fine details.⁴

5. The Denoising Backbone: 2D Diffusion with 3D Priors

This section clarifies a critical distinction: **GeNVS does not use a 3D U-Net**. The diffusion process occurs entirely in the 2D pixel domain. The U-Net is a 2D convolutional network. The "3D" aspect is handled upstream by the feature volume and renderer.

5.1 Architecture: Modified DDPM++ / ADM

The backbone is based on the **Ablated Diffusion Model (ADM)**, also known as DDPM++, which is the standard for high-quality image synthesis (e.g., utilized in OpenAI's guided diffusion papers).

Hyperparameter Configuration (The "90M Parameter" Model): The GeNVS model is designed to be lightweight compared to massive text-to-image models. Based on the analysis of parameter counts (~90M) and input resolution (128x128), the configuration is as follows ⁴:

Hyperparameter	Value	Description
Model Class	2D U-Net	Standard 2D convolutional U-Net.
Input Channels	19	3 (Noisy Target RGB) + 16 (Conditioning Feature Image).
Output Channels	3	Predicted Noise ϵ (or clean image x_0).
Base Channels	128	Width of the initial layer.
Channel Multipliers		Channels scale: 128 $\rightarrow$ 256

		$\rightarrow 256 \rightarrow 256$.
Downsampling Steps	3	Resulting in a bottleneck resolution of $16 \times$.
ResNet Blocks	2	Blocks per resolution level.
Attention Resolutions		Self-attention applied only at the lowest resolutions ($16 \times$ and $8 \times$).
Time Embedding	Sinusoidal	Standard geometric position encoding for time steps t .

Note on Attention: GeNVS restricts self-attention to very low resolutions (16 and 8). This is a crucial efficiency choice. Attention scales quadratically with spatial dimension (N^2). Applying it at 64 or 32 resolution would explode memory usage. Since the geometric structure is already strongly provided by the feature volume conditioning, the U-Net relies more on convolutions (local processing) than global attention to generating texture.

5.2 Conditioning Mechanism: Channel-Wise Concatenation

Unlike Stable Diffusion, which injects conditioning (text) via Cross-Attention layers, GeNVS injects its conditioning (the 3D feature image) via **Concatenation**.

- **Why Concatenation?**
 - **Cross-Attention** is spatially invariant; it tells the model *what* to draw (e.g., "a dog") but not necessarily *where*.
 - **Concatenation** is spatially aligned. It effectively "glues" the 3D feature map to the noisy canvas. The U-Net's convolutional filters can then directly correlate a feature at pixel (u, v) in the conditioning image with the noise at pixel (u, v) in the target. This forces the generated pixels to adhere strictly to the geometry rendered by the feature volume.
- **Implementation:**

```
Python
# PyTorch Pseudo-code
# noisy_input:
# feature_cond: (Upsampled from 64x64)
```

```

UNET_INPUT = torch.cat([noisy_input, feature_cond], dim=1)
# UNET_INPUT shape:
prediction = UNET_2D(UNET_INPUT, timestep)

```

5.3 Noise Scheduling and Prediction Target

- **Prediction Target:** GeNVS typically employs ϵ -**prediction**, predicting the noise added to the image.
- **Noise Schedule:** A linear beta schedule is standard for this resolution, though cosine schedules are often preferred for improved contrast. The snippets imply standard DDPM practices are followed.

6. Implementation Strategy & PyTorch Dynamics

Implementing GeNVS requires orchestrating these components into a unified training loop.

6.1 Data Flow and Training Loop

1. **Input:** Batch of data containing source images, source poses, target images, and target poses.
2. **Geometry Pass (Gradient Checkpointed):**
 - Run Encoder on source images $\rightarrow$ Get Volumes.
 - Run Renderer (Ray Marching) using source volumes and target poses $\rightarrow$ Get Feature Image F (at 64x64).
 - Upsample F to 128x128.
3. **Diffusion Pass:**
 - Sample timestep t .
 - Add noise to ground truth target image I_t to get y_t .
 - Concatenate y_t and F .
 - Run U-Net to predict noise $\hat{\epsilon}$.
4. **Optimization:**
 - Loss = $\text{MSE}(\epsilon, \hat{\epsilon})$.
 - **Backward:** Gradients flow from the loss through the U-Net, then through the concatenation into the Renderer (via F), through the sampling/MLP, and finally into the Encoder. This end-to-end training is what makes the features "geometry-aware."

6.2 Autoregressive Inference Logic

To generate consistent video sequences, GeNVS uses an autoregressive loop.

1. **Frame 1:** Generate I_1 using source I_0 .
2. **Frame 2:** The model now treats I_1 (the generated image) as a *new source view*.
 - Encode $I_0 \rightarrow W_0$.
 - Encode $I_1 \rightarrow W_1$.
 - Render target view for I_2 by aggregating rays from W_0 and W_1 .
3. **Drift Mitigation:** By constantly grounding the generation in the geometric features of previous frames, the model resists the "drift" (hallucinating new, inconsistent objects) that plagues pure video diffusion models.

6.3 Implementation Pitfalls

- **Coordinate Systems:** A major source of failure is the mismatch between the dataset's camera coordinate system (e.g., OpenGL: Right, Up, Back) and the renderer's expectation. PyTorch3D and standard NeRF implementations often differ. Visualizing the raycast origins relative to the frustum is essential for debugging.
- **Memory Management:** The 3D volumes are large. If OOM (Out of Memory) errors occur, consider:
 - Reducing the batch size.
 - Using `torch.utils.checkpoint` for the Encoder/Renderer segment.
 - Reducing the number of depth planes ($D = 64 \rightarrow 48$).
 - Utilizing mixed precision (AMP) for the U-Net (but be cautious with the Volume Rendering, which can be unstable in FP16).

7. Comparative Analysis & Performance

To contextualize the architecture, we compare GeNVS with prominent alternatives.

Feature	GeNVS (Ours)	PixelNeRF	3DiM
Core Representation	Hybrid (3D Feature Volume + 2D Diffusion)	Explicit 3D (Feature Volume + MLP)	Pure 2D (Pose-Conditional Diffusion)

Rendering	Differentiable Feature Rendering	RGB Volume Rendering	None (Direct Image Gen)
Conditioning	Spatial Concatenation	N/A	Cross-Attention / Stochastic
Geometry Consistency	High (Enforced by Volume)	Very High (Hard constraint)	Low (Soft constraint via attention)
Image Quality	High (Generative Texture)	Medium (Blurry in occlusion)	High (Generative)
Inference Cost	Medium (Requires ray marching)	Low (Single pass)	High (Heavy attention)
Parameter Count	~90M (Efficient)	~10-20M	~470M+ (Heavy)

Key Takeaway: GeNVS achieves a "Goldilocks" zone. It is significantly lighter than 3DiM (90M vs 470M parameters) because it doesn't need to learn geometry solely through attention weights; the geometry is explicitly provided by the volume. It produces far sharper images than PixelNeRF because it uses a generative prior to texture the scene rather than minimizing a blur-inducing MSE loss.

8. Conclusion

The GeNVS architecture represents a sophisticated fusion of geometric structure and generative AI. For the engineer implementing this in PyTorch, the complexity lies not in the diffusion model itself—which is a standard 2D U-Net—but in the proper construction of the **differentiable geometric pipeline**.

The system relies on a **ResNet-34 encoder** with a **DeepLabV3+ head** (using learnable upsampling) to lift images into a **16-channel, 64-depth frustum volume**. This volume is rendered via **stratified ray marching** into a feature image, which is then concatenated with noise to condition a **lightweight 2D ADM U-Net**. Understanding that the "3D" in GeNVS refers to the conditioning signal and not the generative convolution kernel is the single most important insight for a successful implementation. By rigorously adhering to the dimensionality and integration details outlined in this report, one can reproduce the system's

ability to synthesize novel views that are both photorealistic and geometrically consistent.

Works cited

1. [2304.02602] Generative Novel View Synthesis with 3D-Aware Diffusion Models - arXiv, accessed January 23, 2026, <https://arxiv.org/abs/2304.02602>
2. {GeNVS}: Generative Novel View Synthesis with {3D}-Aware Diffusion Models} - NVlabs, accessed January 23, 2026, <https://nvlabs.github.io/genvs/>
3. CrossModalityDiffusion: Multi-Modal Novel View Synthesis with Unified Intermediate Representation - CVF Open Access, accessed January 23, 2026, https://openaccess.thecvf.com/content/WACV2025W/GeoCV/papers/Berian_CrossModalityDiffusion_Multi-Modal_Novel_View_Synthesis_with_Unified_Intermediate_Representation_WACVW_2025_paper.pdf
4. Generative Novel View Synthesis with 3D-Aware Diffusion Models - NVlabs, accessed January 23, 2026, <https://nvlabs.github.io/genvs/media/genvs.pdf>
5. Generative Novel View Synthesis with 3D-Aware Diffusion Models - CVF Open Access, accessed January 23, 2026, https://openaccess.thecvf.com/content/ICCV2023/papers/Chan_Generative_Novel_View_Synthesis_with_3D-Aware_Diffusion_Models_ICCV_2023_paper.pdf
6. Modern novel view synthesis algorithms: a survey - SPIE Digital Library, accessed January 23, 2026, <https://www.spiedigitallibrary.org/conference-proceedings-of-spie/13459/134590X/Modern-novel-view-synthesis-algorithms-a-survey/10.1117/12.3052175.full>
7. GeNVS: 3D-Aware Diffusion Models | ICCV 2023 - Stanford Computational Imaging Lab, accessed January 23, 2026, <https://www.computationalimaging.org/publications/genvs/>
8. multi-sensor deep learning for glacier mapping - arXiv, accessed January 23, 2026, <https://arxiv.org/pdf/2409.12034?>
9. arXiv:2303.02165v3 [cs.CV] 29 May 2023, accessed January 23, 2026, <https://arxiv.org/pdf/2303.02165>
10. Enhancing the conversational agent with an emotional support system for mental health digital therapeutics - PMC - NIH, accessed January 23, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC10149869/>